

Gesture Detection Using YOLOv8 on Local Machine

1. Introduction

Real-time gesture detection has become an essential component in modern applications, ranging from human-computer interaction to assistive technologies. Leveraging advanced deep learning frameworks, particularly YOLO (You Only Look Once), enables rapid and accurate identification of gestures in images or video feeds. This report outlines the integration of a trained YOLOv8 model from Google Colab to a local Python 3.8 environment for real-time gesture detection using a laptop's camera.

2. Problem Statement

While cloud platforms like Google Colab offer powerful computational resources for training gesture detection models, deploying these trained models on local systems for real-time inference is often challenging. Users frequently encounter issues such as package compatibility, system resource limitations, and the integration of camera feeds with trained models. This project addresses these challenges by:

- Transitioning a trained YOLOv8 model from Colab to a local Python 3.8 environment.
- Configuring the local system to support real-time camera-based gesture detection.
- Providing comprehensive guidance on package installation and runtime execution.

3. Project Objective

The primary objective of this project is to enable the deployment of a YOLOv8-trained model for real-time gesture detection on a local machine. Specifically, the goals are:

1. To configure a Python 3.8 environment with all necessary dependencies.
2. To integrate a laptop's camera for real-time image capture and processing.
3. To execute the YOLOv8 model for detecting and annotating gestures in the camera feed.

4. Scope of the Project

This project is applicable to:

- Individuals or organizations seeking offline deployment of trained YOLOv8 models for gesture detection.
- Developers working on edge devices with limited internet access but requiring real-time inference capabilities.
- Applications such as assistive technologies, gaming, human-computer interaction, and personal projects leveraging gesture detection.

5. Methodology

5.1 Setting up the Python Environment

To ensure compatibility, the following packages are installed for Python 3.8:

- Ultralytics: Provides the YOLOv8 framework.
- OpenCV: Handles real-time video feed from the laptop's camera.
- Torch and Torchvision: For executing the YOLOv8 model during inference.
- NumPy and Pillow: For efficient image manipulation.

Installation Commands:

bash

```
pip install ultralytics==8.0.28
```

```
pip install opencv-python==4.5.5.64 opencv-python-headless==4.5.5.64
```

```
pip install torch==1.10.2 torchvision==0.11.3 -f https://download.pytorch.org/whl/cpu/torch\_stable.html
```

5.2 Loading the YOLOv8 Model

The trained model file ('best.pt') is loaded using the Ultralytics library. This file, previously trained on Colab, is transferred to the local machine.

Code:

python

```
import cv2  
from ultralytics import YOLO
```

```
model = YOLO('Weights/best.pt')
```

5.3 Real-Time Camera Integration

The OpenCV library facilitates capturing video feed from the laptop's camera. Frames from the video feed are passed to the YOLOv8 model for real-time inference.

Code:

```
python
```

```
import cv2
from ultralytics import YOLO

model = YOLO('Weights/best.pt')

def detect_gestures():
    # Open webcam
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("Error: Cannot access the webcam.")
        return
    print("Press 'q' to quit the real-time detection.")
    while True:
        ret, frame = cap.read()
        if not ret:
            print("Error: Frame not read properly. Exiting...")
            break

        results = model(frame)

        annotated_frame = results[0].plot() # Automatically annotates detections

        cv2.imshow("Gesture Detection", annotated_frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    detect_gestures()
```

5.4 Testing and Verification

- The Python script is executed to ensure the camera feed is operational and the trained model performs gesture detection as expected.
- The script supports multiple gesture detection in real-time.

6. References

1. Ultralytics Documentation: <https://docs.ultralytics.com/>
2. OpenCV Documentation: <https://opencv.org/>
3. PyTorch Installation Guide: <https://pytorch.org/get-started/locally/>
4. YOLOv8 GitHub Repository: <https://github.com/ultralytics/ultralytics>